

Java String, StringBuilder, Multidimensional Arrays,

1. Reverse String — LeetCode 344

Problem Statement

Given a character array, reverse the array in-place.

Constraints

$1 \leq s.length \leq 10^5$; $s[i]$ is a printable character.

Sample Input

```
5
hello
```

Sample Output

```
olleh
```

Explanation

Use two pointers. Swap the first and last characters and move toward the center.

Predefined Java Template

```
import java.util.*;

public class Main {
    static void reverseString(char[] s) {
        // Write your code here
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        char[] s = new char[n];
        for (int i = 0; i < n; i++) s[i] = sc.next().charAt(0);
        reverseString(s);
        for (char c : s) System.out.print(c + " ");
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;
```

```

public class Main {
    static void reverseString(char[] s) {
        int left = 0, right = s.length - 1;
        while (left < right) {
            char temp = s[left];
            s[left] = s[right];
            s[right] = temp;
            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        char[] s = new char[n];
        for (int i = 0; i < n; i++) s[i] = sc.next().charAt(0);
        reverseString(s);
        for (char c : s) System.out.print(c + " ");
        sc.close();
    }
}

```

Test Cases

Input: 5 / h e l l o -> Output: o l l e h

Input: 6 / H a n n a h -> Output: h a n n a H

Time & Space Complexity

O(n) time, O(1) extra space.

2. Valid Palindrome — LeetCode 125

Problem Statement

Determine whether a string is a palindrome after ignoring spaces, punctuation, and case.

Constraints

1 <= s.length <= 2 * 10⁵.

Sample Input

A man, a plan, a canal: Panama

Sample Output

true

Explanation

After removing non-alphanumeric characters and converting to lowercase, the string reads the same forward and backward.

Predefined Java Template

```
import java.util.*;

public class Main {
    static boolean isPalindrome(String s) {
        // Write your code here
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(isPalindrome(s));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {
    static boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left < right) {
            while (left < right && !
Character.isLetterOrDigit(s.charAt(left))) left++;
            while (left < right && !
Character.isLetterOrDigit(s.charAt(right))) right--;
            if (Character.toLowerCase(s.charAt(left)) !=
Character.toLowerCase(s.charAt(right)))
                return false;
            left++;
            right--;
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(isPalindrome(s));
        sc.close();
    }
}
```

```
    }  
}
```

Test Cases

Input: A man, a plan, a canal: Panama -> true

Input: race a car -> false

Input: Madam -> true

Time & Space Complexity

O(n) time, O(1) extra space.

3. Find First Occurrence — LeetCode 28

Problem Statement

Given haystack and needle, return the index of the first occurrence of needle in haystack, or -1.

Constraints

1 <= haystack.length, needle.length <= 10⁴.

Sample Input

```
sadbutsad  
sad
```

Sample Output

```
0
```

Explanation

The substring sad first appears at index 0.

Predefined Java Template

```
import java.util.*;  
  
public class Main {  
    static int findOccurrence(String haystack, String needle) {  
        // Write your code here  
        return -1;  
    }  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        String haystack = sc.nextLine();  
        String needle = sc.nextLine();  
        System.out.println(findOccurrence(haystack, needle));  
        sc.close();  
    }  
}
```

Full Java Solution

```
import java.util.*;

public class Main {
    static int findOccurrence(String haystack, String needle) {
        return haystack.indexOf(needle);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String haystack = sc.nextLine();
        String needle = sc.nextLine();
        System.out.println(findOccurrence(haystack, needle));
        sc.close();
    }
}
```

Test Cases

Input: sadbutsad / sad -> 0

Input: leetcode / code -> 4

Input: hello / xyz -> -1

Time & Space Complexity

Using Java indexOf: $O(n)$ typical for this exercise; $O(1)$ extra space.

4. Longest Common Prefix — LeetCode 14

Problem Statement

Given an array of strings, find the longest common prefix shared by all strings.

Constraints

$1 \leq \text{number of strings} \leq 200$; each string length ≤ 200 .

Sample Input

```
3
flower
flow
flight
```

Sample Output

```
fl
```

Explanation

All strings begin with fl, and no longer prefix is common.

Predefined Java Template

```
import java.util.*;

public class Main {
    static String longestCommonPrefix(String[] strs) {
        // Write your code here
        return "";
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        String[] strs = new String[n];
        for (int i = 0; i < n; i++) strs[i] = sc.next();
        System.out.println(longestCommonPrefix(strs));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {
    static String longestCommonPrefix(String[] strs) {
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (!strs[i].startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        String[] strs = new String[n];
        for (int i = 0; i < n; i++) strs[i] = sc.next();
        System.out.println(longestCommonPrefix(strs));
        sc.close();
    }
}
```

Test Cases

Input: flower, flow, flight -> fl

Input: dog, racecar, car -> ""

Input: interview, internet, internal -> inter

Time & Space Complexity

$O(n*m)$ in the usual bound, where m is prefix length; $O(1)$ extra space excluding returned string.

5. Length of Last Word — LeetCode 58

Problem Statement

Return the length of the last word in a string.

Constraints

$1 \leq s.length \leq 10^4$; letters and spaces only.

Sample Input

Hello World

Sample Output

5

Explanation

Skip trailing spaces, then count characters until the next space.

Predefined Java Template

```
import java.util.*;

public class Main {
    static int lengthOfLastWord(String s) {
        // Write your code here
        return 0;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(lengthOfLastWord(s));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {
```

```

static int lengthOfLastWord(String s) {
    int i = s.length() - 1;
    while (i >= 0 && s.charAt(i) == ' ') i--;
    int count = 0;
    while (i >= 0 && s.charAt(i) != ' ') {
        count++;
        i--;
    }
    return count;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String s = sc.nextLine();
    System.out.println(lengthOfLastWord(s));
    sc.close();
}
}

```

Test Cases

Input: Hello World -> 5

Input: Java Programming -> 11

Input: Hello World -> 5

Time & Space Complexity

O(n) time, O(1) extra space.

6. First Unique Character — LeetCode 387

Problem Statement

Find the index of the first character that appears only once; return -1 if none exists.

Constraints

1 <= s.length <= 10⁵; lowercase English letters.

Sample Input

leetcode

Sample Output

0

Explanation

Count all characters first, then scan the string again for the first frequency of 1.

Predefined Java Template

```
import java.util.*;

public class Main {
    static int firstUniqChar(String s) {
        // Write your code here
        return -1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(firstUniqChar(s));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {
    static int firstUniqChar(String s) {
        int[] frequency = new int[26];
        for (char c : s.toCharArray()) frequency[c - 'a']++;
        for (int i = 0; i < s.length(); i++)
            if (frequency[s.charAt(i) - 'a'] == 1) return i;
        return -1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(firstUniqChar(s));
        sc.close();
    }
}
```

Test Cases

Input: leetcode -> 0

Input: loveleetcode -> 2

Input: aabb -> -1

Time & Space Complexity

O(n) time, O(1) extra space because the frequency array has size 26.

7. Valid Anagram — LeetCode 242

Problem Statement

Determine whether two strings are anagrams.

Constraints

1 <= lengths <= 5*10⁴; lowercase English letters.

Sample Input

anagram
nagaram

Sample Output

true

Explanation

Both strings have identical character frequencies.

Predefined Java Template

```
import java.util.*;

public class Main {
    static boolean isAnagram(String s, String t) {
        // Write your code here
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        String t = sc.nextLine();
        System.out.println(isAnagram(s, t));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {
    static boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] frequency = new int[26];
        for (int i = 0; i < s.length(); i++) {
            frequency[s.charAt(i) - 'a']++;
            frequency[t.charAt(i) - 'a']--;
        }
    }
}
```

```

    }
    for (int count : frequency)
        if (count != 0) return false;
    return true;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String s = sc.nextLine();
    String t = sc.nextLine();
    System.out.println(isAnagram(s, t));
    sc.close();
}
}

```

Test Cases

Input: anagram / nagaram -> true

Input: rat / car -> false

Input: listen / silent -> true

Time & Space Complexity

O(n) time, O(1) extra space.

8. Add Binary — LeetCode 67

Problem Statement

Add two binary strings without converting them directly to numeric types.

Constraints

1 <= lengths <= 10⁴; only 0 and 1.

Sample Input

```

11
1

```

Sample Output

```

100

```

Explanation

Process digits from right to left and maintain a carry. StringBuilder constructs the answer.

Predefined Java Template

```

import java.util.*;

public class Main {
    static String addBinary(String a, String b) {

```

```

        // Write your code here
        return "";
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String a = sc.nextLine();
        String b = sc.nextLine();
        System.out.println(addBinary(a, b));
        sc.close();
    }
}

```

Full Java Solution

```

import java.util.*;

public class Main {
    static String addBinary(String a, String b) {
        StringBuilder result = new StringBuilder();
        int i = a.length() - 1, j = b.length() - 1, carry = 0;
        while (i >= 0 || j >= 0 || carry > 0) {
            int sum = carry;
            if (i >= 0) sum += a.charAt(i--) - '0';
            if (j >= 0) sum += b.charAt(j--) - '0';
            result.append(sum % 2);
            carry = sum / 2;
        }
        return result.reverse().toString();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String a = sc.nextLine();
        String b = sc.nextLine();
        System.out.println(addBinary(a, b));
        sc.close();
    }
}

```

Test Cases

Input: 11 / 1 -> 100
 Input: 1010 / 1011 -> 10101
 Input: 0 / 0 -> 0

Time & Space Complexity

$O(\max(n, m))$ time and $O(\max(n, m))$ space for the result.

9. Add Strings — LeetCode 415

Problem Statement

Add two non-negative integers represented as strings without converting them directly to integer types.

Constraints

$1 \leq \text{lengths} \leq 10^4$; digits 0-9.

Sample Input

456

77

Sample Output

533

Explanation

Add digits from right to left with carry, using `StringBuilder`.

Predefined Java Template

```
import java.util.*;

public class Main {
    static String addStrings(String num1, String num2) {
        // Write your code here
        return "";
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String num1 = sc.nextLine();
        String num2 = sc.nextLine();
        System.out.println(addStrings(num1, num2));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {
    static String addStrings(String num1, String num2) {
        StringBuilder result = new StringBuilder();
        int i = num1.length() - 1, j = num2.length() - 1, carry = 0;
        while (i >= 0 || j >= 0 || carry > 0) {
            int sum = carry;
            if (i >= 0) sum += num1.charAt(i--) - '0';
            if (j >= 0) sum += num2.charAt(j--) - '0';
            result.append(sum % 10);
            carry = sum / 10;
        }
        return result.reverse().toString();
    }
}
```

```

        if (j >= 0) sum += num2.charAt(j--) - '0';
        result.append(sum % 10);
        carry = sum / 10;
    }
    return result.reverse().toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String num1 = sc.nextLine();
    String num2 = sc.nextLine();
    System.out.println(addStrings(num1, num2));
    sc.close();
}
}

```

Test Cases

Input: 456 / 77 -> 533

Input: 11 / 123 -> 134

Input: 999 / 1 -> 1000

Time & Space Complexity

$O(\max(n,m))$ time and space.

10. String Compression — LeetCode 443

Problem Statement

Compress consecutive repeated characters in-place and return the new length.

Constraints

$1 \leq \text{chars.length} \leq 2000$.

Sample Input

```

7
a a b b c c c

```

Sample Output

Length: 6; Compressed: a 2 b 2 c 3

Explanation

aaa becomes a3, bb becomes b2, and ccc becomes c3.

Predefined Java Template

```
import java.util.*;

public class Main {

    static String compress(String str) {
        //write code
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        System.out.println(compress(str));

        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {

    static String compress(String str) {

        StringBuilder result = new StringBuilder();

        int i = 0;

        while (i < str.length()) {

            char current = str.charAt(i);
            int count = 0;

            // Count consecutive occurrences
            while (i < str.length() && str.charAt(i) == current) {
                count++;
                i++;
            }

            // Add character
            result.append(current);

            // Add count only if greater than 1
            if (count > 1) {
                result.append(count);
            }

            i++;
        }

        return result.toString();
    }
}
```

```

        public static void main(String[] args) {

            Scanner sc = new Scanner(System.in);

            String str = sc.nextLine();

            System.out.println(compress(str));

            sc.close();

        }
    }

```

Test Cases

Input: a a b b c c c -> Length 6, a 2 b 2 c 3

Input: a a a a a a -> Length 2, a 6

Input: a b c d e -> Length 5, a b c d e

Time & Space Complexity

O(n) time and O(1) auxiliary space.

11. Reverse Words in a String — LeetCode 151

Problem Statement

Reverse the order of words and remove extra spaces.

Constraints

1 <= s.length <= 10⁴.

Sample Input

the sky is blue

Sample Output

blue is sky the

Explanation

Trim the input, split on one-or-more spaces, and append words from right to left using StringBuilder.

Predefined Java Template

```

import java.util.*;

public class Main {
    static String reverseWords(String s) {
        // Write your code here
        return "";
    }
}

```



```

        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            String s = sc.nextLine();
            System.out.println(reverseWords(s));
            sc.close();
        }
    }
}

```

Full Java Solution

```

import java.util.*;

public class Main {
    static String reverseWords(String s) {
        String[] words = s.trim().split("\\s+");
        StringBuilder result = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            result.append(words[i]);
            if (i != 0) result.append(" ");
        }
        return result.toString();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(reverseWords(s));
        sc.close();
    }
}

```

Test Cases

Input: the sky is blue -> blue is sky the

Input: hello world -> world hello

Input: Java is very powerful -> powerful very is Java

Time & Space Complexity

$O(n)$ time and $O(n)$ space.

12. Richest Customer Wealth — LeetCode 1672

Problem Statement

Each row is a customer and each column is a bank. Find the maximum total wealth of any customer.

Constraints

1 <= rows, columns <= 50; values are positive integers.

Sample Input

```
2 3
1 2 3
3 2 1
```

Sample Output

6

Explanation

Sum each row and track the largest sum.

Predefined Java Template

```
import java.util.*;

public class Main {
    static int maximumWealth(int[][] accounts) {
        // Write your code here
        return 0;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int rows = sc.nextInt(), cols = sc.nextInt();
        int[][] accounts = new int[rows][cols];
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                accounts[i][j] = sc.nextInt();
        System.out.println(maximumWealth(accounts));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {
    static int maximumWealth(int[][] accounts) {
        int max = 0;
        for (int i = 0; i < accounts.length; i++) {
            int sum = 0;
            for (int j = 0; j < accounts[i].length; j++) sum +=
accounts[i][j];
            max = Math.max(max, sum);
        }
    }
}
```

```

        return max;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int rows = sc.nextInt(), cols = sc.nextInt();
        int[][] accounts = new int[rows][cols];
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                accounts[i][j] = sc.nextInt();
        System.out.println(maximumWealth(accounts));
        sc.close();
    }
}

```

Test Cases

Input:

3 2

1 2 3

3 2 1

Output:6

Input:

3 2

1 5

5 7

3 5

Output:10

Time & Space Complexity

$O(\text{rows} \times \text{cols})$ time, $O(1)$ extra space.

13. Matrix Diagonal Sum — LeetCode 1572

Problem Statement

Find the sum of both diagonals of a square matrix without counting the center twice.

Constraints

$1 \leq n \leq 100$.

Sample Input

```
3
1 2 3
4 5 6
7 8 9
```

Sample Output

```
25
```

Explanation

Add `matrix[i][i]` and `matrix[i][n-1-i]`. If they refer to the same center cell, add it only once.

Predefined Java Template

```
import java.util.*;

public class Main {
    static int diagonalSum(int[][] matrix) {
        // Write your code here
        return 0;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                matrix[i][j] = sc.nextInt();
        System.out.println(diagonalSum(matrix));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {
    static int diagonalSum(int[][] matrix) {
        int n = matrix.length, sum = 0;
        for (int i = 0; i < n; i++) {
            sum += matrix[i][i];
            if (i != n - 1 - i) sum += matrix[i][n - 1 - i];
        }
        return sum;
    }

    public static void main(String[] args) {
```

```

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                matrix[i][j] = sc.nextInt();
        System.out.println(diagonalSum(matrix));
        sc.close();
    }
}

```

Test Cases

Input: 3x3 matrix above -> 25

Input: [[1,2],[3,4]] -> 10

Input: [[5]] -> 5

Time & Space Complexity

$O(n)$ time, $O(1)$ extra space.

14. Transpose Matrix — LeetCode 867

Problem Statement

Return the transpose of a matrix by converting rows into columns.

Constraints

$1 \leq \text{rows, columns} \leq 1000$.

Sample Input

```

2 3
1 2 3
4 5 6

```

Sample Output

```

[1, 4]
[2, 5]
[3, 6]

```

Explanation

Create a cols-by-rows result and set $\text{result}[j][i] = \text{matrix}[i][j]$.

Predefined Java Template

```

import java.util.*;

public class Main {
    static int[][] transpose(int[][] matrix) {
        // Write your code here
    }
}

```

```
        return new int[0][0];
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int rows = sc.nextInt(), cols = sc.nextInt();
        int[][] matrix = new int[rows][cols];
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                matrix[i][j] = sc.nextInt();
        int[][] result = transpose(matrix);
        for (int[] row : result)
            System.out.println(Arrays.toString(row));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {

    static int[][] transpose(int[][] matrix) {

        int rows = matrix.length;
        int cols = matrix[0].length;

        int[][] result = new int[cols][rows];

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                result[j][i] = matrix[i][j];
            }
        }

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int rows = sc.nextInt();
        int cols = sc.nextInt();

        int[][] matrix = new int[rows][cols];

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        int[][] result = transpose(matrix);

        // Print transpose
        for (int i = 0; i < result.length; i++) {
            for (int j = 0; j < result[0].length; j++) {
                System.out.print(result[i][j]);

                if (j < result[0].length - 1) {
                    System.out.print(" ");
                }
            }
        }
    }
}
```

```

        }
        System.out.println();
    }

    sc.close();
}
}

```

Test Cases

Input: `[[1,2,3],[4,5,6]]` -> `[[1,4],[2,5],[3,6]]`

Input: `[[1,2],[3,4]]` -> `[[1,3],[2,4]]`

Time & Space Complexity

$O(\text{rows} \times \text{cols})$ time and $O(\text{rows} \times \text{cols})$ result space.

15. Reshape Matrix — LeetCode 566

Problem Statement

Reshape a matrix into $r \times c$ if the number of elements stays the same; otherwise return the original matrix.

Constraints

$1 \leq \text{rows}, \text{columns}, r, c \leq 100$.

Sample Input

```

2 2
1 2
3 4
1 4

```

Sample Output

```

[1, 2, 3, 4]

```

Explanation

Flatten conceptually using one index. If total element counts match, map source index to destination row and column.

Predefined Java Template

```

import java.util.*;

public class Main {
    static int[][] matrixReshape(int[][] matrix, int r, int c) {
        // Write your code here
        return matrix;
    }
}

```



```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int rows = sc.nextInt(), cols = sc.nextInt();
    int[][] matrix = new int[rows][cols];
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            matrix[i][j] = sc.nextInt();
    int r = sc.nextInt(), c = sc.nextInt();
    int[][] result = matrixReshape(matrix, r, c);
    for (int[] row : result)
        System.out.println(Arrays.toString(row));
    sc.close();
}
}

```

Full Java Solution

```

import java.util.*;

public class Main {

    static int[][] reshape(int[][] matrix, int newRows, int newCols) {

        int rows = matrix.length;
        int cols = matrix[0].length;

        // Check whether total number of elements is same
        if (rows * cols != newRows * newCols) {
            return matrix;
        }

        int[][] result = new int[newRows][newCols];

        int index = 0;

        // Traverse original matrix
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {

                int newRow = index / newCols;
                int newCol = index % newCols;

                result[newRow][newCol] = matrix[i][j];

                index++;
            }
        }
    }
}

```

```

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Original matrix size
        int rows = sc.nextInt();
        int cols = sc.nextInt();

        int[][] matrix = new int[rows][cols];

        // Read matrix
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        // New size: 1 x 4
        int newRows = sc.nextInt();
        int newCols = sc.nextInt();

        int[][] result = reshape(matrix, newRows, newCols);

        // Print result
        for (int i = 0; i < newRows; i++) {
            for (int j = 0; j < newCols; j++) {
                System.out.print(result[i][j] + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}

```

Test Cases

Input: 2x2 [[1,2],[3,4]], reshape 1x4 -> [1,2,3,4]
 Invalid reshape returns original matrix.

Time & Space Complexity

$O(\text{rows} \times \text{cols})$ time and $O(r \times c)$ result space.

16. Rotate Image — LeetCode 48

Problem Statement

Rotate an $n \times n$ matrix 90 degrees clockwise in-place.

Constraints

$1 \leq n \leq 20$; matrix is square.

Sample Input

```
3
1 2 3
4 5 6
7 8 9
```

Sample Output

```
[7, 4, 1]
[8, 5, 2]
[9, 6, 3]
```

Explanation

Transpose the matrix, then reverse every row.

Predefined Java Template

```
import java.util.*;

public class Main {
    static void rotate(int[][] matrix) {
        // Write your code here
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                matrix[i][j] = sc.nextInt();
        rotate(matrix);
        for (int[] row : matrix)
            System.out.println(Arrays.toString(row));
        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;
```

```

public class Main {
    static void rotate(int[][] matrix) {
        int n = matrix.length;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }
        for (int i = 0; i < n; i++) {
            int left = 0, right = n - 1;
            while (left < right) {
                int temp = matrix[i][left];
                matrix[i][left] = matrix[i][right];
                matrix[i][right] = temp;
                left++;
                right--;
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                matrix[i][j] = sc.nextInt();
        rotate(matrix);
        for (int[] row : matrix)
            System.out.println(Arrays.toString(row));
        sc.close();
    }
}

```

Test Cases

Input: 3x3 matrix above -> [7,4,1], [8,5,2], [9,6,3]

Input: [[1,2],[3,4]] -> [[3,1],[4,2]]

Time & Space Complexity

$O(n^2)$ time, $O(1)$ extra space.

17. Spiral Matrix — LeetCode 54

Problem Statement

Return all elements of a matrix in spiral order.

Constraints

$1 \leq \text{rows}, \text{columns} \leq 10$.

Sample Input

```
3 3
1 2 3
4 5 6
7 8 9
```

Sample Output

```
[1, 2, 3, 6, 9, 8, 7, 4, 5]
```

Explanation

Maintain top, bottom, left, and right boundaries and shrink them after each direction.

Predefined Java Template

```
import java.util.*;

public class Main {

    static void spiralMatrix(int[][] matrix) {

        //write code here
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int rows = sc.nextInt();
        int cols = sc.nextInt();

        int[][] matrix = new int[rows][cols];

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        spiralMatrix(matrix);

        sc.close();
    }
}
```

Full Java Solution

```
import java.util.*;

public class Main {

    static void spiralMatrix(int[][] matrix) {

        int top = 0;
        int bottom = matrix.length - 1;

        int left = 0;
        int right = matrix[0].length - 1;

        while (top <= bottom && left <= right) {

            // 1. Left to Right
```

```

        for (int j = left; j <= right; j++) {
            System.out.print(matrix[top][j] + " ");
        }
        top++;

        // 2. Top to Bottom
        for (int i = top; i <= bottom; i++) {
            System.out.print(matrix[i][right] + " ");
        }
        right--;

        // 3. Right to Left
        if (top <= bottom) {
            for (int j = right; j >= left; j--) {
                System.out.print(matrix[bottom][j] + " ");
            }
            bottom--;
        }

        // 4. Bottom to Top
        if (left <= right) {
            for (int i = bottom; i >= top; i--) {
                System.out.print(matrix[i][left] + " ");
            }
            left++;
        }
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int rows = sc.nextInt();
    int cols = sc.nextInt();

    int[][] matrix = new int[rows][cols];

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = sc.nextInt();
        }
    }

    spiralMatrix(matrix);

    sc.close();
}

```

```
    }  
}
```

Test Cases

Input: 3x3 -> [1,2,3,6,9,8,7,4,5]

Input: 1x4 [1,2,3,4] -> [1,2,3,4]

Input: 3x1 [1,2,3] -> [1,2,3]

Time & Space Complexity

O(rows*cols) time and O(rows*cols) output space.

28. Private Fields + Getters/Setters

Problem Statement

Create a Student class with private name and marks. Use public getters and setters to access them.

Constraints

Name is a string; marks is an integer.

Sample Input

Pavani

95

Sample Output

Name: Pavani

Marks: 95

Explanation

Private fields cannot be accessed directly from outside the class. Public getters and setters provide controlled access.

Predefined Java Template

```
import java.util.*;  
  
class Student {  
    private String name;  
    private int marks;  
  
    // Write getters and setters here  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        String name = sc.nextLine();
```



```

        int marks = sc.nextInt();

        Student student = new Student();

        // Set values
        // Print values

        sc.close();
    }
}

```

Full Java Solution

```

import java.util.*;

class Student {
    private String name;
    private int marks;

    public void setName(String name) {
        this.name = name;
    }

    public void setMarks(int marks) {
        this.marks = marks;
    }

    public String getName() {
        return name;
    }

    public int getMarks() {
        return marks;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String name = sc.nextLine();
        int marks = sc.nextInt();

        Student student = new Student();
        student.setName(name);
        student.setMarks(marks);

        System.out.println("Name: " + student.getName());
        System.out.println("Marks: " + student.getMarks());
    }
}

```

```
        sc.close();  
    }  
}
```

Test Cases

Input: Pavani / 95 -> Name: Pavani, Marks: 95

Input: Rahul / 88 -> Name: Rahul, Marks: 88

Time & Space Complexity

O(1) time per getter/setter; O(1) extra space.